

Hostel Management & GatePass — SRS v2 (Final Specification)

Status: Requirements locked, ready for development.

Supersedes: Original SRS v1. All ambiguities resolved via requirement-gathering sessions.

1. Scope & Deployment Model

1.1 Tenancy

- **Single university, 8 hostels** (mixed boys/girls). One shared identity system (university email + roll number as unique keys).
- One database, one deployment, one Razorpay merchant account (owned by the university's finance office).
- Schema threads `hostel_id` through all tables so multi-institution expansion remains possible later.

1.2 v1 Inclusions

- Visual bed booking (one student, one bed) with soft locks, payment, warden verification
- Annual room-change window (reuses booking flow, no payment collected)
- Mid-semester vacate workflow
- Gatepass system with scheduled enforcement (6 PM weekdays / all weekend), QR validation, guard app
- Fee management: **hostel fees + miscellaneous dues only** (lump sum, no installments)
- Bulk user provisioning + self-registration with Super Admin approval
- Data Integrity module (duplicate merge/delete) with permanent audit trail
- Super Admin web panel, Warden web + mobile, Student mobile app, Guard scanner app

1.3 v1 Exclusions (backlog)

- Academic fee ledgers (read-only/manual entries at most)
- Recurring gatepasses
- Visitor gatepasses (stays on paper register)
- Guard shift rostering
- Biometric app unlock
- Installment/partial payments
- Student self-service data export
- Multi-institution tenancy

2. Scale & Environment Assumptions

Parameter	Design Value
Students	~2,000–2,500 (architecture headroom to 10x)
Hostels	8 × 3 floors × ~30–35 rooms, double/triple sharing
Staff (excl. Super Admin)	~20 (multi-hostel wardens incl. acting wardens, guards)
Peak concurrency	~500 simultaneous users on bed grid during booking week

Parameter	Design Value
Peak scan rate	~50–80 scans per 30-min window at gate rush
Devices	Students/guards use own phones; wardens own phone + web dashboard

3. User Roles & RBAC

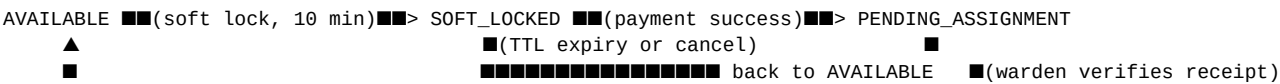
Capability	Student	Warden	Guard	Super Admin
Book / change bed	Own only	—	—	—
View bed grids	All (availability only)	All	—	All + edit floor plans
Request gatepass	Own only	—	—	—
Approve gatepass	—	Own hostel(s) only	—	Any (override, logged)
View gatepass queue	Own only	All hostels	—	All
Scan / validate QR	—	—	✓	—
Manual entry fallback	—	—	✓ (reason code required)	—
Levy misc fines	—	Own hostel students	—	Any + waive/void
Verify bed assignment	—	✓ (physical receipt check)	—	—
Bed status changes	—	✓	—	✓
Fee entry creation	—	Misc fines only	—	Hostel fee structure
Open booking / room-change windows	—	—	—	✓
Assign acting wardens (date-ranged)	—	—	—	✓
Approve self-registrations	—	—	—	✓
RBAC provisioning	—	—	—	✓ (MFA)
Holiday calendar	—	—	—	✓
Data export (security/fee logs)	—	—	—	✓
Duplicate profile merge/delete	—	—	—	✓

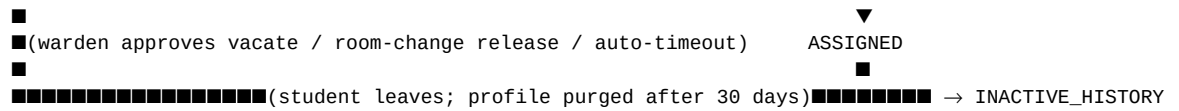
Key role rules:

- Super Admin = the single overall incharge person. MFA mandatory.
- Wardens **see all** gatepasses/beds but **approve only for their assigned hostel(s)**.
- Warden↔hostel assignment is date-ranged: Super Admin can attach an acting warden to any hostel for N days (leave coverage). One warden account may cover multiple hostels.
- Guards: minimal scanner-only access, own authenticated session per shift (no formal rostering).

4. Bed Booking System

4.1 Bed State Machine





4.2 Rules

- **One student, one bed.** No group leader / multi-bed cart.
- **Soft lock TTL: 10 minutes.** If payment is in progress or failed midway, the lock extends **once** for an additional 10 minutes (tracked via Razorpay order status polling). After that, the bed releases automatically and broadcasts unlock via WebSocket.
- **Roommate preference:** optional free-text field at booking time ("prefer room with [roll number]"). Non-binding; warden sees it during physical verification and may honor it (adjacent beds, same room). No state-machine impact.
- **Payment:** lump sum via Razorpay hosted checkout (UPI/cards/net banking). On success → PENDING_ASSIGNMENT + PDF receipt generated.
- **Pending-assignment window** = the booking period (default 1 week). Configurable per booking round by Super Admin (start date, end date, verification deadline). On timeout: bed releases; **no refund** (university policy: all sales final).
- **Warden verification:** student presents receipt in person; warden dashboard shows **system-verified payment badge** (the PDF alone is NOT trusted — it's editable). Warden confirms identity → flips to ASSIGNED. This is a human-in-the-loop checkpoint and is intentional.

4.3 Room-Change Window (annual reshuffle)

- Super Admin opens a configurable window (dates set per round).
- During the window, ASSIGNED students see a **Change Room/Bed** action: releases current bed (returns to pool) → full booking flow (soft lock, etc.) → **no payment collected** (uni does not re-charge).
- Warden physically re-verifies.
- Outside the window, change requests are disabled.

4.4 Vacate

- Student requests vacate → **warden approval required** (implies room inspection) → bed returns to pool. No refund.
- Temporary absence: no state change; bed remains ASSIGNED.

4.5 UI

- CSS Grid/Flexbox floor maps (no canvas). Clickable available beds; locked/occupied greyed out with live WebSocket updates; visible countdown timer on held beds.

5. Gatepass System

5.1 Enforcement Schedule

- Pass **not required** on weekdays before 6 PM (class hours) — gates unmonitored, no scanning, no logging.
- Pass **required:** after 6 PM weekdays, and **all day weekends**.
- Holiday calendar maintained by Super Admin (university-wide or per hostel). Holidays behave like weekend (pass required all day) — configurable.

5.2 Lifecycle

DRAFT → PENDING → APPROVED → ACTIVE (exit scanned) → RETURNED → EXPIRED
 ■ (never exits before validity end)
 ■ ■■■> NO_SHOW → (warden may cancel)
 ■■■> REJECTED → (student may re-request)
 Extensions: APPROVED/ACTIVE → EXTENSION_REQUESTED → (warden) → APPROVED(new time) / DENIED

- Student submits: reason, expected exit, expected return. Warden may modify return time before approving.
- On approval: **single static QR** generated in-app. Valid for the approved window; **auto-expires exactly 60 minutes after the return scan is logged**.
- **Single round-trip only**. Re-entry requires a new pass. No recurring passes in v1.

5.3 QR Design

- QR payload: **cryptographically signed token (JWT)** containing pass ID, student ID, validity window. Guard device validates **locally and instantly** (no network wait).
- **Anti-replay**: same QR scanned twice in the same direction → loud warning screen.
- On scan, guard's device prominently displays: **photo (large, unavoidable), name, roll number, hostel, room, pass validity window**. Guard's duty includes face-to-photo match — this is the primary defense against QR screenshot sharing.
- Revocation: pass cancelled by warden → push revocation to guard devices when online; JWT validity kept short enough that stale offline validations self-limit.

5.4 Late Alerts

- Default: 30-minute grace after approved return time → alert fires.
- **Alert criteria fully customizable per warden** (grace duration, time-of-day rules).
- Alert goes to all wardens of that hostel (+ Super Admin).
- **No automatic punitive action** — everything at warden's discretion. (Late fines may be levied manually via misc dues, if warden chooses.)

5.5 Emergency Extensions

- Student off-campus messages warden in-app requesting time extension → warden approves/denies → return time updated, QR validity extended, alert clock recalculates.

5.6 Guard App & Connectivity

- Minimalist scanner app; shift context captured from authenticated login.
- **Offline mode (queue-and-sync)**: JWT validates locally with zero connectivity; scan logs stored in an **encrypted on-device queue**, auto-sync on reconnect; visible "pending sync" badge.
- **Ultra-low-bandwidth mode by default**: student photos cached on device while online; no heavy payloads over cellular.
- **Dead-phone fallback**: guard searches roll number → sees photo + active passes → manually marks entry/exit. **Mandatory reason code; logged permanently; warden notified** for every manual entry.

6. Fee Management

6.1 Scope & Ledgers

- **Hostel Fees + Miscellaneous Dues only**. Academic fees excluded from v1.
- When Super Admin opens a booking round, hostel fee is set per hostel/room-type; system auto-generates a due for every student on bed assignment.

6.2 Rules

- **Lump sum only.** No installments, no partial payments. One payment → one receipt → one assignment.
- **No refunds** (university policy). Exception path for system errors only, handled by Super Admin manually via Razorpay dashboard.
- **Misc fines/dues:** wardens levy against students in their hostel (late-return, damage); students may dispute in-app (message thread to warden); Super Admin can waive/void. *(Assumption — validate with warden office.)* All fine actions audit-logged.
- **Reminders:** push + email before due date.

6.3 Receipts

- Instant PDF receipt on payment success; downloadable anytime. Receipts stored in S3; required as convenience document at warden verification (system badge is authoritative).

7. Onboarding & Identity

- **Bulk import (primary):** Super Admin uploads CSV/Excel roster (start of year); maps fields; accounts auto-provisioned; welcome emails sent. **Official ID photos come from university records (management's duty); students do not self-upload photos.**
- **Self-registration (secondary):** student signs up with university email + roll number → **"Pending Verification"** (restricted) → **Super Admin approves** after matching against roster.
- **Duplicate handling:** roll number + email are unique keys. Data Integrity module flags conflicts; Super Admin executes one-click **Merge** (consolidates logs/fees) or **Delete**. Every merge/delete permanently audit-logged.
- **Departure:** profile data purged **30 days after student leaves hostel**. Assigned bed released via vacate flow.

8. Privacy, Retention & Audit

8.1 Retention Schedule (decided by app owners)

Data	Retention
Gatepass & scan logs	2 years
Financial / transaction records	8 years
Profile data	Until 30 days post-departure → purged
Admin audit logs	10 years
All manual/exception actions (guard manual entries, merges, overrides)	Permanent, immutable — regardless of who performed them

8.2 Privacy

- Biometric unlock: **excluded from v1.**
- Student photos used only for guard verification; cached on guard devices; purged with profile.
- DPDP Act, 2023 alignment: minimal data collection, consent at registration, manual data-export process via Super Admin in v1.

8.3 Mandatory Audit Trail

Every sensitive action is logged immutably with actor + timestamp + before/after:

- Warden approvals/denials/timing modifications
- Bed status changes (locked → assigned, releases)
- Guard scans AND manual fallback entries
- Fine levies/waivers
- RBAC changes, acting-warden assignments
- Profile merges/deletes
- Data exports

9. Notifications

Event	Channel	Recipients
Pass approved/rejected/modified	Push	Student
Late return (post-grace)	Push (+email for wardens)	All wardens of hostel + Super Admin
Extension request	Push	Warden(s)
Lock expiry warning (T-2 min)	Push	Student
Payment success/failure	Push + email	Student
Fee due reminder	Push + email	Student
Manual guard entry	Push	Warden(s)
Booking window opening/closing	Push + email	Students

- No SMS in v1. Quiet-hours policy: none for security alerts; marketing-type alerts suppressed 10 PM–7 AM.

10. Non-Functional Requirements

Category	Target
Uptime	99.5%
Scan validation (online)	< 1s end-to-end
Bed-lock broadcast	< 500ms to all clients
Concurrent capacity	500 simultaneous booking users; 10x architectural headroom
Backups	Daily; RTO < 4 hours; RPO 24h
Accessibility	WCAG 2.1 AA for web dashboards
App support	Last 2 major OS versions (iOS/Android)

11. Technical Architecture

- **Web:** React.js (Super Admin + Warden dashboards)
- **Mobile:** React Native (Student, Warden, Guard apps)
- **Backend:** Node.js + Express, REST APIs for CRUD/billing
- **Real-time:** WebSockets (Socket.io) — bed locks, live grid updates, scan sync, push alerts
- **DB:** PostgreSQL (ACID for financials; relational gatepass/security logs)
- **Offline sync:** guard device encrypted local queue (SQLite/encrypted store)
- **QR:** signed JWT payload, local validation, rotation-free

- **Payments:** Razorpay hosted checkout (test mode in dev); webhook reconciliation for stuck/failed payments; idempotent order handling
- **Deployment:** Docker on AWS; S3 for photos/floor plans/PDF receipts; auto-scaling for booking-week load

11.1 WebSocket Event Catalog (initial)

bed:lock, bed:unlock, bed:assigned, grid:update, gatepass:new, gatepass:approved, gatepass:revoked, scan:validated, scan:synced, alert:late, extension:requested, fine:levied

11.2 Failure Handling

- Payment stuck in created state > 15 min → auto-investigate via Razorpay Orders API; lock expires per TTL rules.
- WebSocket disconnect → client falls back to polling for grid state; resync on reconnect.
- Device offline > sync queue → oldest-first encrypted sync; guard sees pending count.

12. Go-Live Plan

1. Development + Razorpay test-mode integration
2. Internal QA incl. concurrency tests (500 simultaneous locks on same bed → exactly 1 success), QR expiry race tests, webhook idempotency tests
3. Import real roster + floor plans; staff training (wardens, guards)
4. **2-week parallel run with the paper register** — scans logged alongside paper; discrepancies investigated daily
5. Go live; paper register retired

13. Backlog (post-v1)

Recurring passes · Visitor passes · Academic fee integration · Installments · Biometric unlock · Guard shift rostering · Multi-institution tenancy · Self-service data export · Fine auto-escalation rules